

| Backend Engineer

☎ 010-9101-5429 | ✉ digle117@gmail.com | 📄 [GitHub github.com/PreAgile](https://github.com/PreAgile) | 🌐 astro-paper-23v.pages.dev

📄

📄 📄 · 📄 📄 📄 · 📄 📄 📄 📄 📄 📄 📄 📄 5📄 📄 📄 📄 📄 📄 . NestJS 📄 SaaS 📄 📄 📄 📄 · 📄 📄 · 📄 📄 · 📄 📄 · 📄 📄 📄 **Java/Kotlin/Spring** 📄 📄 📄 JVM 📄 📄 📄 📄 📄 . 📄 📄 📄 📄 ADR(Architecture Decision Record)📄 PR 📄 📄 📄 📄 📄 📄 .

📄 📄 📄 6📄 📄 📄 📄 📄 SaaS 📄 📄 📄 📄 📄 , 📄 **API** 📄 **20📄** · 📄 📄 📄 **100📄** · 📄 📄 **12📄** 📄 📄 📄 📄 📄 📄 · 📄 📄 · 📄 📄 📄 . **Kotest organization** 📄 📄 (6 PR 📄), **LINE Armeria** (1 PR 📄), **Spring Batch** · **Spring Cloud Gateway** 📄 📄 📄 📄 .

📄 📄 📄

📄	Before → After	📄
📄 📄 📄	📄 800📄 → 90📄 📄 (88.75% ↓)	📄 IP 📄 📄 , 📄 📄 📄
📄 📄 📄 📄	70% → 98%	📄 📄 📄
📄 📄 (📄 6📄)	99.2%	📄 📄 📄 FIFO + lease TTL
📄 📄 📄 📄	0📄 / 6📄	Idempotency-Key + 📄 📄 📄
📄 📄 📄 webhook race	📄 + 📄	📄 📄 📄 📄 📄

$r_{\text{J}} = \text{📄 📄} / \text{📄 📄 📄} \cdot r_{\text{J}} = \text{JVM/Spring 📄 📄 📄 📄 📄} . \text{📄 📄 📄 📄 📄 📄} .$

📄 📄

Production — 📄 📄 📄 📄 📄

- TypeScript · NestJS · Node.js · TypeORM
- MySQL / Aurora · PostgreSQL · Redis (Cluster · Lua · 📄 📄 · Token Bucket)
- RabbitMQ (Quorum Queue · DLX) · 📄 📄 📄 application-level retry
- Docker · Docker Compose · Traefik 📄 📄 · AWS ECS · 📄 📄 📄 📄
- Prometheus · Grafana · APM (span tagging) · Slack Webhook · JFR

JVM / Spring Re-design — 📄 📄 📄 📄 · 📄 📄 📄

- Java · Kotlin · Spring Boot · Spring Batch · JPA · QueryDSL
- Redisson · Resilience4j (Circuit Breaker · Retry · Bulkhead · RateLimiter · TimeLimiter)
- Kafka (Outbox · CDC · Consumer Group) · ShedLock · OAuth2

Evidence — 📄 · 📄 📄 · 📄 📄

- Testcontainers · JUnit 5 · Kotest · Pytest · Jest · nGrinder
- Kotest organization 📄 · Armeria 📄 · Spring Batch 📄 📄

📖 📖

JVM 📖 📖 📖 — 📖 7 · 📖 📖 + **PR** 📖 📖.

[Kotest](#) — **Kotlin** 📖 📖 📖 (4.7k stars)

2026.03 📖 📖 sksamuel 📖 📖 📖 **organization** 📖 📖, 📖 📖 6 PR 📖.

- 📖 📖 Test Metadata Public API 📖 📖 (+355/-16, 5 📖 16h 📖 📖) — [issue #5103](#), [PR #5905](#)
- 📖 📖 data class 📖 📖 diff — [issue #2545](#), [PR #5835](#)
- JSON Schema anyOf / oneOf 📖 📖 — [issue #4463](#), [PR #5807](#)
- JSON Matchers 📖 📖 Json 📖 📖 📖 — [issue #4601](#), [PR #5795](#)
- @OnlyInputTypes 📖 📖 📖 📖 — [issue #5589](#), [PR #5789](#)
- shouldHaveSingleElement 📖 📖 📖 — [issue #5755](#), [PR #5756](#)

[Armeria](#) — **LINE** 📖 **Java** 📖 📖 **RPC** 📖 📖 (5.1k stars)

- RequestContextExporter BuiltInProperty 📖 + Logback MDC export 📖 📖 (+339/-5, **MERGED** 2026.04) — [issue #4403](#), [PR #6683](#)

[Spring Batch](#) — **Spring** 📖 📖 📖 📖 📖 (2.9k stars)

📖 📖 📖, PR 📖 📖.

- fault-tolerant step chunk scanning 📖 📖 4📖 📖 📖 — [issue #3946](#) (📖 📖 "documentation issue" 📖 📖)
- conditional flow decider order 📖 — [issue #4478](#) (📖 📖 "this is a bug" 📖)

[Spring Cloud Gateway](#) — **Spring** 📖 **reactive API gateway**

📖 📖 📖, 5.1.x 📖 📖 📖.

- Apache HttpClient5 📖 📖 📖 📖 — [issue #3311](#) (📖 📖 📖 + 📖 📖)
- API Versioning Predicates 📖 📖 📖 — [issue #4024](#) (📖 📖 📖 📖 📖)

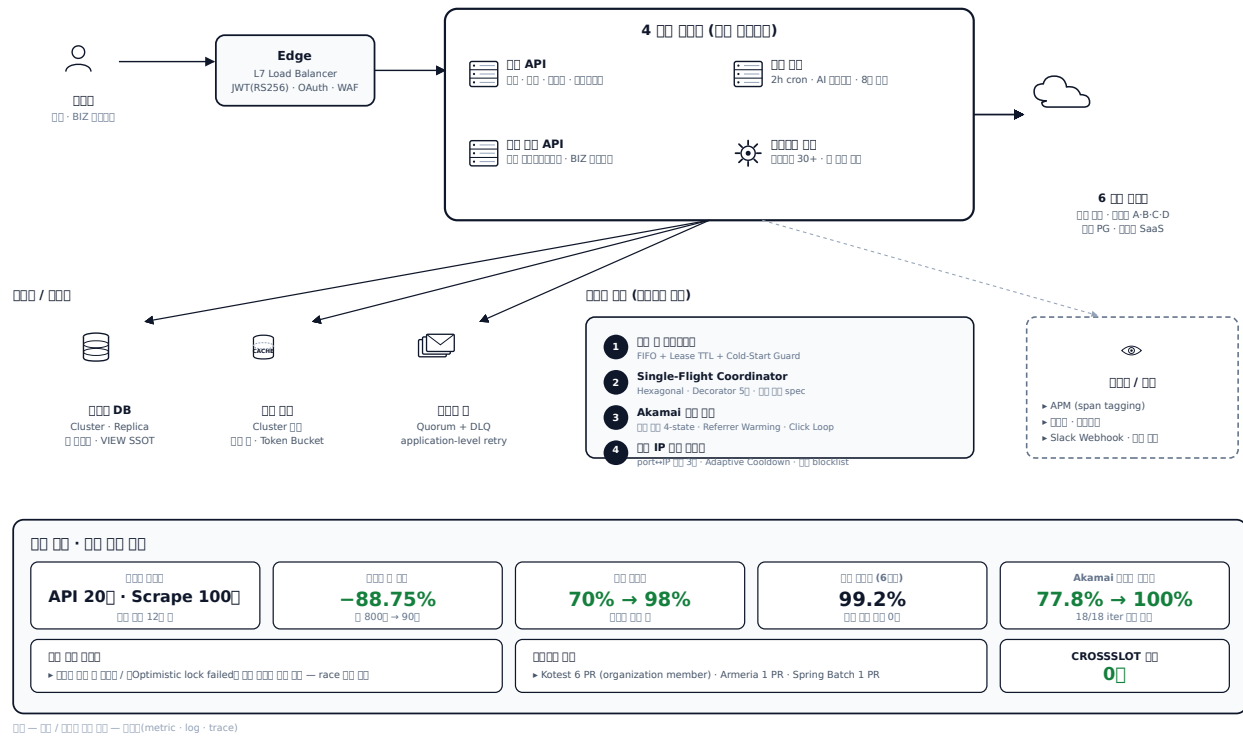
📖 — 📖(**Lemong**) | | 📖 **2024.11** ~ 📖

📖 📖

📖 📖 📖 📖 📖 📖 SaaS 「 / 📖 Biz📖 📖 📖 📖. 4 📖 📖 (📖 API · 📖 📖 · 📖 📖 · 📖 📖) 📖 📖 📖, 📖 📖 · 📖 📖 (Cluster) · 📖 DB · 가 📖 📖 📖 📖

A1. 如何构建 SaaS — 架构概览

4 个组件 · 6 个组件 · 1 个组件 · 1 个组件



监控仪表盘 / 日志仪表盘 — 指标(metric) · 日志(log) · 追踪(trace)

如何构建 SaaS

如何构建 SaaS + 如何构建 SaaS + 如何构建 JVM/Spring 如何构建 SaaS。

2. **DB UNIQUE** — `imp_uid + merchant_uid + status` · 1 dedupe window · fail-open
row

3. — `READY → PAID / FAILED / CANCELLED × action_type` · (PAID `FAILED`)
row

4. **Lua** — `GET → DEL` · TTL `ABA`

reconciliation cron — 3 inconsistency

- **schedule** — DB `K < N` PG `getSchedulesByBillingKey` → 4-way
- — `GROUP BY user_id HAVING COUNT > 1` SQL
- **admin override orphan** — `active.end_date > scheduled.schedule_at + 1month` 4 JOIN SQL

— `(findOne((user, coupon)) idempotent return)` · `(QueryRunner)` ·
`schedule` `amountWithUserCoupons fallback`

()

- 「 merchant_uid=... 」 — race +
- SETNX + Lua DEL helper 8 (promotional upgrade · change plan · verify code ·) helper
- 09:10 KST Slack — / CS
- 0 (UNIQUE + findOne), update 0

— 4 + reconciliation cron · (Slack)

Spring — `@Transactional propagation` · `Redisson watchdog` · `JPA @Version`

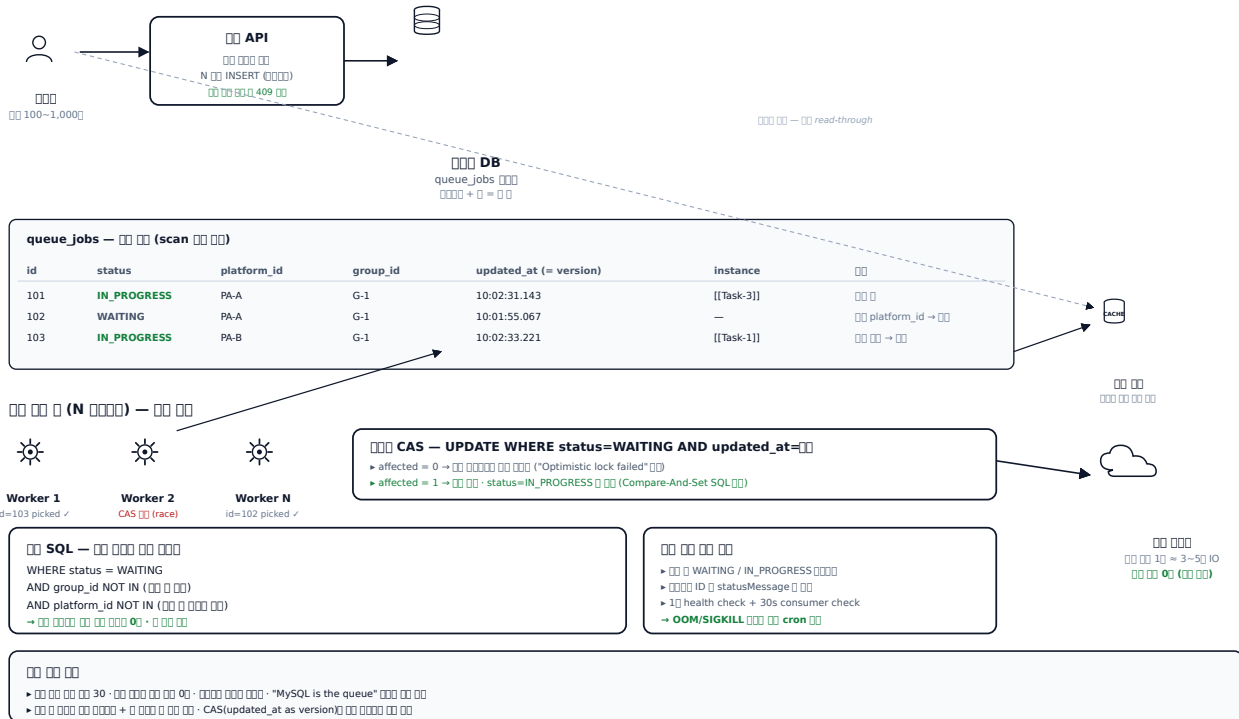
`OptimisticLockException` · `ER_DUP_ENTRY → @RestControllerAdvice` · `ShedLock cron` ·

`QueryDSL` ([commerce-comment-platform-be](https://github.com/commerce-comment-platform-be))

DB + CAS + RabbitMQ

B1. 如何 DB 与 CAS 如何 如何 如何

「MySQL is the queue」 · 가 CAS · ∈ 0000 00 0000 · 00 00 00 00



DB + Hierarchical

□□ — □□ □□□ □□□ □ □□ □□□ □□□□□.

- 1000 1000 1000 (IO ≈ 3~5 IO) — 1000 100~1,000」 1000 1000. HTTP 1000 1000 1000 Load Balancer 50 10000 1000, 1000 10000 in-memory queue 1000 1000 1000, 1000 1000 1000 1000 worker 2000 1000 1000 1000 1000 1000 → 1000 1000 1000
- 2000 **cron** 1000 1000 — 1000 1000 1000 1000 **13,127** 1000 / **5,719** 1000 1000 / **13,699 task** 1000. 1000 10000 1000 1000 1000 1000 captcha / 10000 / 1000 1000. 1000 1000 ThreadPool 1000 1000 1000 1000 1000 1000 1000 1000 = 1000 1000, 1000 1000 1000 1000 → 2000 cron 1000 1000 1000

Our work is orthogonal to the work on fire-and-forget [10] and stateful [11, 12, 13] message queues. BullMQ is not a fire-and-forget message queue, nor is it a stateful message queue. BullMQ is a message queue that is designed to be used in a fire-and-forget manner, but it is also designed to be used in a stateful manner.

□□ — □ □□□ □□□□□.

(a) **DB** **CAS** — **updated_at** **version** **ORM** = **CAS** (**Compare-And-Set**) **SQL**. **affected = 0** **SQL** **platform** **id NOT IN**

(b) 每个平台 / 每个商店的 **hierarchical** 数据 — (platform_id, password) 数据对映射到 ThreadPool 中的 worker thread 中。每个 worker thread 中维护一个列表。每个平台商店的 _deactivate_shops 列表中的每个元素 (平台商店) — 都是一个元素。

모든 (모든 것) — 모든 것의 모든 것의 모든 것 AAS 모든 것의 모든 것 → **RabbitMQ** 모든 것의 모든 것의 모든 것. 모든 것 / 7개 stateful 모든 것 fire-and-forget 모든 것의 모든 것의 모든 것, DB 모든 것의 모든 것 source of truth 모든 것의 모든 것 fan-out 모든 것의 모든 것의 모든 것.

모든 (모든 것)

- 모든 것의 모든 것 30, 모든 것의 모든 것 + 모든 것의 모든 것의 모든 것
- 모든 것 Job N modified by another instance (optimistic lock failed) 모든 것 — race 모든 것의 모든 것의 모든 것
- 모든 것의 모든 것 0, 모든 것의 모든 것
- 모든 것의 모든 것 13,127 모든 것 200 cron 모든 것, 모든 것 platform_id 모든 것 0
- 모든 것의 모든 것의 모든 것 · 모든 것의 모든 것 (모든 것)

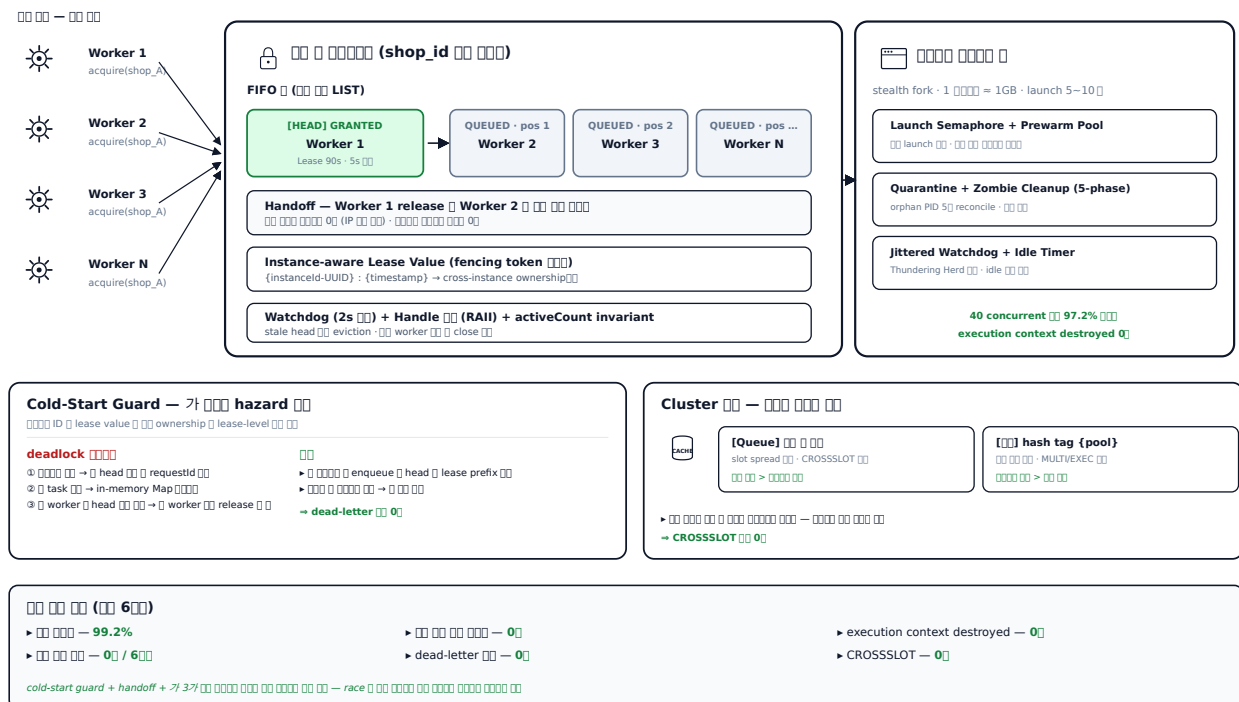
모든 것 — RDS Queue 모든 것 / 모든 것 SQL CAS 모든 것 / 모든 것의 모든 것 (모든 것의 모든 것 + health check) 모든 것. hierarchical 모든 것의 모든 것 + 모든 것의 모든 것 dict 모든 것. 모든 것 RabbitMQ 모든 것의 모든 것 ADR 모든 것.

Spring 모든 것 — JPA @Version 모든 것 + OptimisticLockException (CAS) · Spring Batch + ShedLock + QuerydslZeroOffset Reader · Resilience4j Bulkhead.semaphore (모든 것의 모든 것) + CircuitBreaker (모든 것의 모든 것) · Kafka Outbox / Consumer 모든 것 + DLQ (모든 것의 모든 것 [commerce-batch-orchestrator](#) 모든 것 RabbitMQ↔Kafka 모든 것의 모든 것)

모든 것의 모든 것 + 모든 것의 모든 것 — **30+** 모든 것의 모든 것 **race** · 모든 것의 모든 것의 모든 것

C1. 모든 것의 모든 것 — FIFO + Lease TTL + Cold-Start Guard

30+ 모든 것의 모든 것 race 모든 것 4 layer 모든 것 + Handoff 모든 것 + 모든 것 30



모든 것의 모든 것 + Cold-Start Guard

□□ — 30+ worker □ □□ □□ □□ □□ □□ □□ race + □□ □□ □□ □□ □□ □□ □ □ □ □□ □□ □□ □□.

□□ race:

- □□ □□ □ page □□ navigation = □□□□ □□□□ execution context destroyed
- □□ □□ □□ □□ □ □ = □□ □□ IP □□ □□ □□ (1~24□□ □ □□ □□ □□ □□)
- □□□□ □□□□ 1 □□□□ ≈ 1GB+, launch 5~10□ — race □ □□□·□□□□ N□ □□
- □□ □□ Cluster □□□□ □□ □□ □□ □ □□ □ CROSSSLOT
- 가 □□□□ hazard: **cold-start deadlock** — 가 □□□□ □□□□ in-memory Map □ □□□□□ □□ □□ □ head □ □ □□□□ requestId. □□□□ worker □ □ head □□□□ □□ □□□□ □ worker □ □□□ □□□□ release □ □

□□ □□ □□:

- □□ □□ □□□□ □ □□ □□□□ □□ □□□□ □□□□ 99% □□
- □□ □□□□ □□ □□□□ pending → verified → blocked □ □□ □□□ □□□, □ □□□ 4 □□ silent □□□□ □□ □□□□ password retry □ □□ □□□□

□□ — □ □□ □□□□ □□□ □□□ □□□ □□ + □□ □□□ □□□ □□□ □□.

(a) □□ □ □□□□□ — **4 layer** □□□:

- FIFO □ (□□ □□ LIST) · per-request lease (90s TTL · 5s □□)
- **Instance-aware lease value** = {instanceId-UUID} : {timestamp} (Martin Kleppmann fencing token □□ □□□ — cross-instance ownership □□□□ lease-level □□)
- Handoff pattern (release □ □□ □ entry □ □□ □□ □□□)
- □□□□ 3□ — forceRelease (□□ kill switch) · forceTerminate (activeCount □□ 0) · evictStaleHead (2s □□ stale head LREM)
- □□ □□ Cluster CROSSSLOT — queue □ □□ □ □□ (□□ □□ □□), □□ □□□□ hash tag {pool} (□□□□ □□ □□) □□ □□□ □□□ □□ □□

(b) □□ □□□ □□□ □□ — □□ □□ □□□:

- □□ □□ 4-state □□□□ (□□ / □□ / challenge / □□) — 「 □ □□ challenge > □□ > □□□ □□□□ □ □□□ 4 □□ silent □□□ □□ □□
- □□ □□□ service □ spec □ SSOT □□ □□□ □□ (discriminated union □□)
- Referrer Warming — □□ URL □□ referrer chain □□, 15□ human-like mouse jitter □□ → sensor telemetry □□ → □□ □□ □□
- □□ □□ helper □ main loginResult / Quick Retry / Prewarm Swap 3 □□□□ □□ □□ (silent □□ □□)

□□ (□□ □□ · □□ 6□□)

- □□ □□□ **99.2%**, □□ □□ □□ □□□ **0□**
- □□ □□ □□ **0□**, execution context destroyed □□ **0□**
- □□□□ □□□ □ dead-letter □□ **0□** (cold-start guard □□)
- CROSSSLOT □□ **0□** (hash tag / □□ □ □□ □□ □□ □□)
- □□ □□□ □□□ □□□ **77.8% → 100%** (Referrer Warming □□ · 18/18 iter □□ □□ · PR □□□□ iter × worker □□□□ □□ □□ □□ □ □□□□ □□□□)

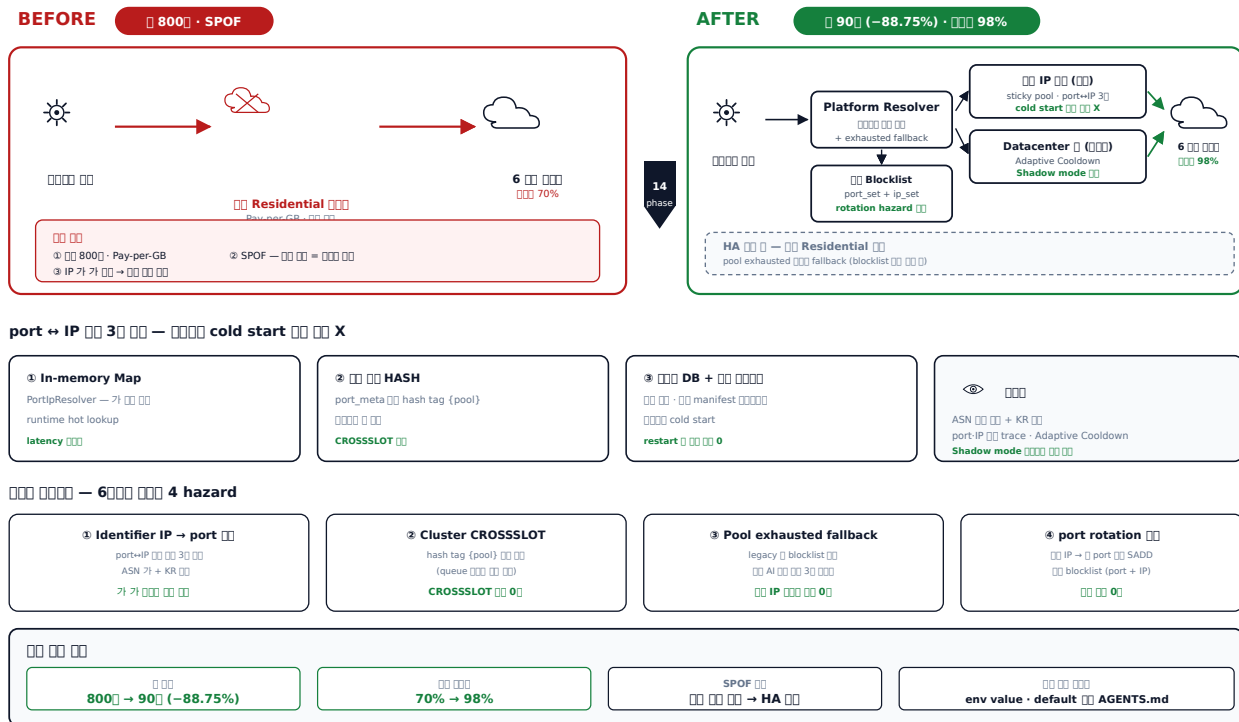
□□ □□ — □□ □ □□□□□ (4 layer + □□□□ 3□) □□ · □□ · □□. □□ □□ □□□□ □□ □□ spec / service SSOT □□ □□□□.

Spring — Redisson RFairLock (FIFO + lease auto-extend) · Kotlin Coroutines Mutex.withLock per-key in-process layer · sealed class discriminated union · Resilience4j Bulkhead
 (commerce-external-gateway-kt spec 7 spec)

IP — + **88.75%**

C2. IP — **88.75%** · **70% → 98%**

14 phase phased rollout — port→IP 3 · Adaptive Cooldown · blacklist · pool exhausted fallback



IP Before / After

— Residential · SPOF ·

- / **SPOF** — 800 · =
- **Identifier** — Datacenter port IP 가 가 (IP) stable identifier (port) 가
- **Cluster CROSSLOT** — · STREAM · blacklist MULTI/EXEC
- **port rotation** — IP port mapping port blacklist

— 6 14 phase phased rollout. phase → → phase

4:

- **port→IP 3** (In-memory Map + HASH + DB + manifest) — cold start
- **blacklist** — port_set + ip_set. allocator port IP lookup → ip_set hit port port_set 가. port rotation hazard

- **Adaptive Cooldown** — 5 outcome (success / block / timeout / networkError / authError / siteChange / unknownError) × consecutiveFailures × latency → health OK, Shadow mode → OK OK OK OK OK
- **Pool exhausted → legacy fallback** — NaverIspPoolExhaustedError throw → legacy path OK, legacy → selectPortRespectingBlocklist → OK IP skip

OK OK AGENTS.md OK OK — env value OK · OK · deprecated alias OK OK OK default OK OK. fallback OK OK OK **AI** OK OK **3 follow-up** OK OK OK PR OK OK OK OK.

OK (OK OK)

- OK OK **800 → 90 OK (88.75% OK)** — OK OK OK
- OK OK **70% → 98%** — OK OK OK OK OK
- OK OK SPOF OK (HA OK OK OK), OK OK OK OK
- OK IP OK legacy OK OK OK OK **0**
- CROSSLOT OK **0** — hash tag {pool} OK OK

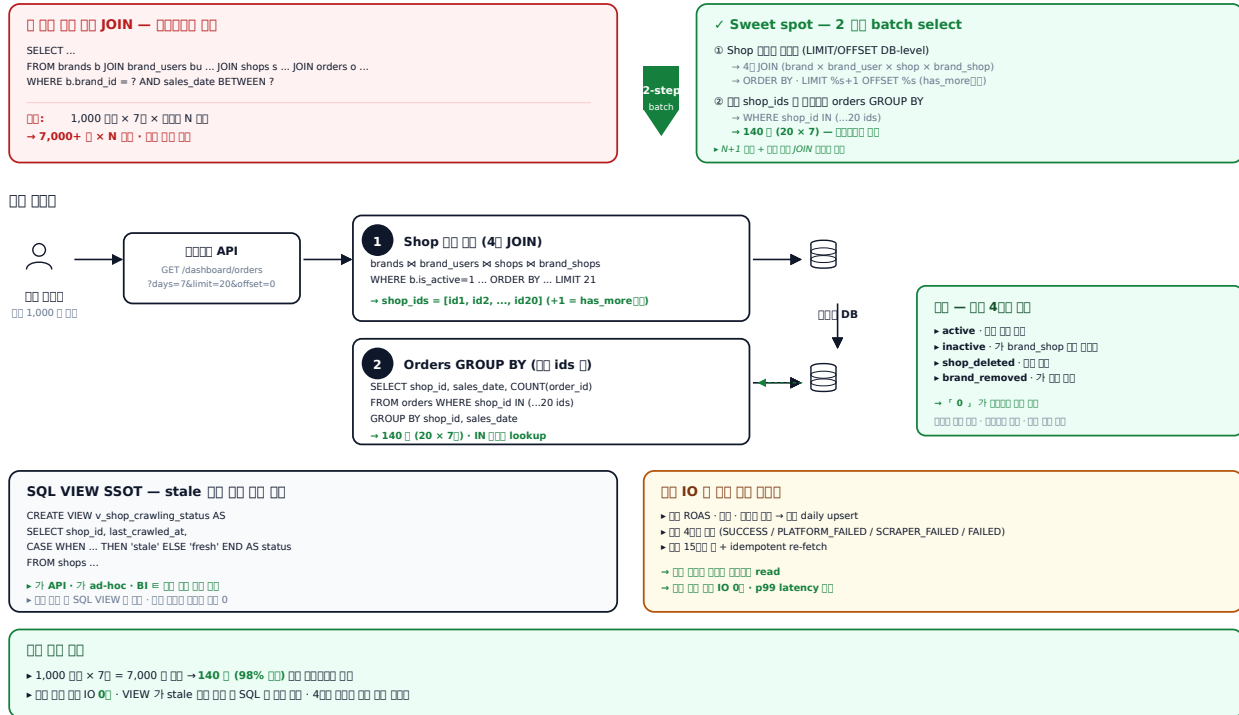
OK OK — 14 phase phased rollout OK phase OK ADR OK · OK · OK · OK OK OK OK OK. OK AI OK OK follow-up OK.

Spring OK — Spring Cloud Gateway + Resilience4j Retry / CircuitBreaker / Bulkhead OK · WebClient ExchangeFilterFunction OK proxy resolver OK · @ConfigurationProperties + @Validated OK env OK (OK OK [commerce-external-gateway-kt](#))

BIZ 問題 — 200 行 + SQL VIEW + 高 IO 行 行 行

B3. BIZ 問題 — 2 行 行 + SQL VIEW SSOT + 高 IO 行 行 行

行 行 → 行 行 1,000 × 70 行 行 — N+1 行 行 + 行 行 JOIN 行 行 sweet spot



BIZ 問題 200 行 + SQL VIEW SSOT

行 行 — 行 行 行 行 100 行 行 1,000 行 行 行 行 行 行 行 行.

行 行 行 行 行 行 行: 行 行 行 行 70 行 行 行 行, 行 行 ROAS / 行 行 / 行 行, 行 行 70, 行 行 行 行 (stale) 行 行, 行 行 行 行.

- 行 行 JOIN 行 行 — brand → ... → orders 行 行 行 行 行 行 行 行 (1,000 × 7 × 行 行 N)
- 行 行 行 行 / 行 行 IO 行 行 行 行 行 行 行 行 行 行
- ORM 行 行 行 行 40 行 行 + dynamic where + order by + LIMIT 行 行 行 行 行 行

行 行 — 「N+1 行 行 行 行 JOIN 行 行 sweet spot 行 行 2 行 行 batch select」 + 「SQL VIEW 行 行 SSOT」 + 「IO 行 行 行 行 行 行 行 行 行 3 行 行.

- **Step 1** — shop 行 行 行 行 (brand × brand_user × shop × brand_shop 40 行 行 · LIMIT %s+1 OFFSET %s has_more 行 行 行 行 DB-level pagination)
- **Step 2** — 行 行 20 shop_ids 行 行 行 行 orders GROUP BY → 140 行 (20 × 7) 行 行 行 行 行 行
- 行 行 400 行 行 (active / inactive / shop_deleted / brand_removed) — 行 行 行 行 行 行 0」 行 行 行 行
- stale 行 行 行 行 SQL VIEW (v_shop_crawling_status) 行 行 行 行 API · 行 行 ad-hoc · BI 行 行 行 行 行 行 行 (single source of truth)
- 行 行 / 行 行 行 行 daily upsert + 400 行 行 (SUCCESS / PLATFORM_FAILED / SCRAPER_FAILED / FAILED) — 行 行 行 行 行 行 行 行 read

行 行 (行 行 行 行)

- 1,000 × 7 = 7,000 行 行 → 140 行 (98% 行 行) 行 行 行 行 行 行

- 100ms IO 0% · p99 latency 100ms
- VIEW 100ms stale — stale 100ms SQL 100ms
- 150ms + 100ms idempotent re-fetch 100ms 100ms

100ms — 200ms batch select 100ms · VIEW SSOT 100ms · 400ms ADR 100ms 100ms.

Spring 100ms — Spring Data JPA + QueryDSL 100ms 100ms · @Subselect SQL VIEW 100ms · @EntityGraph N+1 100ms · Cursor 100ms 100ms (100ms 100ms [commerce-comment-platform-be](https://commerce-comment-platform-be.com))

100ms 100ms — 100ms 100ms

100ms 100ms 100ms 100ms 100ms 100ms (100ms 100ms 100ms / 100ms 100ms):

- **Single-Flight Coordinator** — Hexagonal Architecture (Port-Adapter) + Decorator 50ms 100ms. 100ms 100ms N 100ms → 100ms 100ms 100ms, 100ms 100ms 100ms. 100ms 100ms 700ms spec 100ms 100ms. 100ms 80+ 100ms · 22 new unit spec.
- **SafeCron** 100ms — 100ms 100ms cron 100ms 100ms 100ms. NestJS DI 100ms (setModuleRef) 100ms 100ms 100ms 100ms 100ms + Slack 100ms + 100ms 100ms 100ms. 140ms 100ms.
- 100ms 100ms — 100ms 1,000+ 100ms 100ms 100ms. SELECT FOR UPDATE 100ms + 100ms 100ms 100ms (@Transactional). 100ms 100ms / 100ms row 100ms 100ms.
- 100ms 100ms 100ms 100ms — 200+ 100ms 100ms 100ms 100ms (1 100ms ≈ 1GB+ RAM). 100ms 100ms 100ms + 100ms 100ms ASN 100ms 100ms 100ms 100ms **32 100ms / 512 GB RAM** 100ms 100ms **8 100ms** 100ms. Traefik 100ms 100ms + 5-step 100ms 100ms 100ms.
- 100ms 100ms 100ms — 100ms → AI → TOCTOU 4 100ms → API Key/JWT 100ms 100ms 100ms. 100ms 100ms 100ms + Layered Idempotency 100ms 100ms.
- 100ms 100ms 100ms — 6 100ms 100ms 100ms (Shannon 100ms · zlib 100ms · n-gram · 100ms run) + 가 100ms 100ms Bell + Polarity Amplification + tanh box) + 100ms 100ms OR 100ms.

JVM/Spring 100ms 100ms — Node.js 100ms 100ms Java/Kotlin/Spring 100ms 100ms

50ms 100ms Node.js / NestJS 100ms 100ms 100ms 100ms 100ms · 100ms 100ms · 100ms 100ms · 100ms 100ms 100ms Java/Kotlin/Spring 100ms 100ms 100ms 100ms 100ms, 100ms 100ms **ADR (Architecture Decision Record — 100ms 100ms 100ms · 100ms · 100ms 100ms 100ms 100ms 100ms)** 100ms 100ms 100ms.

100ms 100ms NestJS 100ms 100ms 100ms, 100ms 100ms Java/Kotlin/Spring/JPA 100ms 100ms ADR 100ms 100ms 100ms.

100ms 100ms **ADR 49** · 100ms **47** · 100ms 100ms **17** 100ms (100ms 100ms — 100ms 100ms 100ms).

[commerce-comment-platform-be](https://commerce-comment-platform-be.com) — 100ms 100ms (Java / Spring Boot / JPA)

- Stage 0 (raw JDBC) → Stage 5 (DDD / MSA) 100ms · Lombok 100ms · record / sealed 100ms · OSIV=false
- **ADR-006** 100ms 100ms **4** 100ms — idempotency_key 100ms + 4 100ms + Redis SETNX 100ms + DB UNIQUE 100ms 100ms + 100ms 100ms reconciliation 40ms 100ms (Stripe / 100ms 100ms)
- **ADR-004** 100ms 100ms — Redisson watchdog vs SET NX+Lua 100ms 100ms Redisson 100ms
- **ADR-BE-007** 100ms 100ms — REPEATABLE READ 100ms + RC 100ms (EXP-06 4 100ms phantom 100ms)
- **ADR-BE-008** 100ms 100ms — 100ms 100ms 100ms 100ms 100ms 100ms (EXP-09b 9 100ms 100ms 100ms)

- **ADR-BE-009 No-Offset Pagination** — Cursor `row constructor` (EXP-07 1M OFFSET 171ms → Cursor)
- `GET_LOCK` (EXP-02 4), `@Version` (EXP-03 silent Lost Update 74) / `OptLockEx` 86, EXP-04 N+1 4-depth 121 `prepStmt` → JOIN FETCH 1, EXP-13 dirty checking 132x `readOnly`, EXP-14 `IDENTITY=saveAll` batch

[commerce-batch-orchestrator](#) — Kafka / Outbox

- **ADR-005 Outbox** → **Debezium CDC** — Dual-Write
- **ADR-MQ-006 RabbitMQ vs Kafka** — RabbitMQ classic queue replay 가 · HOL blocking · Consumer Group Kafka + RabbitMQ
- **ADR-MQ-007 Spring Batch Reader 4** — `QuerydslZeroOffsetItemReader`
- **ADR-MQ-008 Consumer + DLQ + retry topic** — At-least-once + = Effectively-Once
- **ADR-MQ-009 CooperativeStickyAssignor (KIP-429)** — Rebalance stop-the-world
- — E-MQ-01 Direct publish → E-MQ-02 Outbox → E-MQ-03 Debezium CDC, E-MQ-04~07 Reader 4 100 row, E-MQ-08/09 RabbitMQ ↔ Kafka throughput, E-MQ-10 Eager vs Cooperative Sticky rebalance

RabbitMQ `ADR-MQ-006` .

[commerce-external-gateway-kt](#) — (Kotlin / Coroutines / Resilience4j)

- **ADR-002 Coroutines vs Virtual Threads** — Coroutines ()
- **ADR-007 Circuit Breaker** — 50% / 70% / 90% 70% (CB-1/2/3)
- **ADR-SCR-008 Single-Flight Coordinator** — TS 7 Kotlin Coroutines Mutex / Deferred . SF-2 99.9% coalescing
- **ADR-SCR-009 3-Layer** — Memory L1 → Redis L2 → L3 (NAVER)
- **ADR-SCR-010 Bulkhead** — Semaphore vs FixedThreadPool
- **ADR-SCR-011 Resilience4j** — Bulkhead → CB → Retry → TimeLimiter → RateLimiter (Retry CB CB N)
- — EXP-WC-01 WebClient .block() EventLoop pinning 1/10 →

— (I-BRICKS) | | 2021.05 ~ 2024.11

, , .

- **EBS (Kafka · Apache Nifi · Elasticsearch)** — Kafka producer / consumer + Nifi flow + Elasticsearch . INSERT → Kafka + Nifi 20 50%. (Kafka — JVM)
- **(Elasticsearch + Logstash)** — RDB → ES ,
- — React / Redux / SCSS UI, KWAG 2.1



F-Lab Java Backend Mentoring | 2024.01 ~ 2024.07

Meta ☐ ☐ ☐ ☐ ☐ ☐. ☐ ☐ ☐, ☐ ☐ ☐, ☐ ☐ ☐ ☐ ☐ ☐, CQRS, ☐ ☐ ☐ ☐.

☐ ☐ ☐ ☐ ☐ ☐